



PYTHON NOTES — LISTS AND LIST OPERATIONS



TOPIC: LISTS AND LIST OPERATIONS

A **list** is a Python data structure used to store multiple values in one variable.

Lists are useful when a program needs to keep a collection of related items, such as names, contacts, scores, tasks, or products.

A list can store multiple values, and the items can be changed after the list is created.

1. CREATING A LIST

A list is created using square brackets `[]`.

```
contacts = []
```

This creates an empty list.

We can also create a list with values:

```
names = ['Chris', 'Joyce', 'Belle']
```

2. LIST ITEMS

Each value stored in a list is called an **item** or **element**.

Example:

```
fruits = ['Apple', 'Banana', 'Mango']
```

The list contains three items:

```
Apple  
Banana  
Mango
```

3. ACCESSING LIST ITEMS

List items are accessed using their index.

Python uses **zero-based indexing**.

That means the first item has index `0`.

```
fruits = ['Apple', 'Banana', 'Mango']  
  
print(fruits[0])  
print(fruits[1])  
print(fruits[2])
```

Output:

```
Apple  
Banana  
Mango
```

Index positions

```
Apple   → index 0  
Banana  → index 1  
Mango   → index 2
```

4. CHANGING LIST ITEMS

Lists are mutable, which means their items can be changed.

Example:

```
fruits = ['Apple', 'Banana', 'Mango']  
fruits[1] = 'Orange'  
  
print(fruits)
```

Output:

```
['Apple', 'Orange', 'Mango']
```

The item at index `1` was replaced.

5. `.append()`

`.append()` adds one new item to the **end** of a list.

Example:

```
contacts = []  
  
contacts.append('Chris')  
contacts.append('Joyce')  
  
print(contacts)
```

Output:

```
['Chris', 'Joyce']
```

Important

Each time `.append()` is used, the new item is placed at the end of the list.

6. `.remove()`

`.remove()` removes an item by its **value**.

Example:

```
contacts = ['Chris', 'Joyce', 'Belle']  
  
contacts.remove('Joyce')  
  
print(contacts)
```

Output:

```
['Chris', 'Belle']
```

The item 'Joyce' was removed.

Important

`.remove()` uses the item value, not its index.

7. `.pop()`

`.pop()` removes an item by its **index**.

Example:

```
contacts = ['Chris', 'Joyce', 'Belle']  
  
contacts.pop(1)  
  
print(contacts)
```

Output:

```
['Chris', 'Belle']
```

The item at index 1 was removed.

Difference

```
remove(value) → remove using the item itself  
pop(index)    → remove using the item's position
```

8. `len()`

`len()` tells us how many items are in a list.

Example:

```
contacts = ['Chris', 'Joyce', 'Belle']  
  
print(len(contacts))
```

Output:

```
3
```

`len()` is especially useful when a program needs to know how many items are stored.

9. CHECKING IF A LIST IS EMPTY

One way to check whether a list has no items is:

```
if len(contacts) == 0:  
    print('There are no contacts.')
```

This is useful in programs that need to behave differently when a list is empty.

10. `enumerate()`

`enumerate()` allows us to loop through a sequence while also receiving a counter/index.

Example:

```
contacts = ['Chris', 'Joyce', 'Belle']  
  
for number, name in enumerate(contacts, 1):  
    print(number, '.', name)
```

Output:

```
1 . Chris
2 . Joyce
3 . Belle
```

The `1` tells `enumerate()` to start counting from `1` instead of `0`.

This is especially useful when displaying numbered lists to users.

11. LOOPING THROUGH A LIST

A `for` loop can go through each item in a list.

```
names = ['Chris', 'Joyce', 'Belle']

for name in names:
    print(name)
```

Output:

```
Chris
Joyce
Belle
```

12. CHECKING IF AN ITEM IS IN A LIST

We can use `in` to check whether a value exists in a list.

Example:

```
contacts = ['Chris', 'Joyce', 'Belle']

if 'Joyce' in contacts:
    print('Contact found')
```

This is useful when searching a list.

13. LISTS WITH USER INPUT

Lists can store information entered by the user.

Example:

```
names = []

name = input('Enter a name: ')
names.append(name)

print(names)
```

A program can use a loop to keep accepting more values.

14. LISTS WITH `while True`

A list can be combined with a `while True` loop to create a program that keeps accepting items.

Example:

```
names = []

while True:
    name = input('Enter a name or type done: ')

    if name == 'done':
        break

    names.append(name)

print(names)
```

This combines:

- Lists
- `.append()`
- `while True`
- `if`
- `break`

- `input()`

15. MY `lists.py` PROJECT — CONTACT MANAGER

This is the larger Lists project I created.

```
import os

def clear_screen():
    os.system('cls')

contacts = []

while True:
    clear_screen()

    print('1. Add contact')
    print('2. View contacts')
    print('3. Delete contacts')
    print('4. Count contacts')
    print('5. Exit')
    print('Your option must be a number')

    choice = int(input('Choose an option: '))

    if choice == 1:
        name = input('Enter a contact name: ')
        contacts.append(name)

    elif choice == 2:
        if len(contacts) == 0:
            print('There are no contacts to view')
        else:
            for number, name in enumerate(contacts, 1):
                print(number, '.', name)

    elif choice == 3:
        if len(contacts) == 0:
            print('There are no contacts to delete')
        else:
            for number, name in enumerate(contacts, 1):
                print(number, '.', name)

            name = input('Type the contact or number you want to delete: ')
```



```
    if name.isdigit():
        num = int(name)

        if 1 <= num <= len(contacts):
            contacts.pop(num - 1)
            print('Contact successfully deleted.')
        else:
            print('Invalid contact number')

    elif name in contacts:
        contacts.remove(name)
        print('Contact successfully deleted.')

    else:
        print('Contact not found.')

elif choice == 4:
    print('You have', len(contacts), 'contacts.')

elif choice == 5:
    print('Goodbye!')
    break

else:
    print('Your choice is invalid!')
```

16. WHAT MY CONTACT MANAGER DOES

The program creates an empty list:

```
contacts = []
```

The program then repeatedly displays a menu.

Option 1 — Add contact

```
name = input('Enter a contact name: ')
contacts.append(name)
```

A new contact is added to the list.

Option 2 — View contacts

The program first checks whether the list is empty:

```
if len(contacts) == 0:
```

If it is empty, it displays a message.

Otherwise, it uses:

```
enumerate(contacts, 1)
```

to display the contacts with numbers.

Option 3 — Delete contacts

The program displays the contacts first.

Then the user can enter either:

- A contact number
- A contact name

If a number is entered, the program uses `pop()`:

```
contacts.pop(num - 1)
```

The `- 1` is necessary because the user sees numbers starting at `1`, while list indexes start at `0`.

If a name is entered, the program uses `remove()`:

```
contacts.remove(name)
```

Option 4 — Count contacts

The program uses:

```
len(contacts)
```

to count how many contacts are stored.

Option 5 — Exit

The program prints a goodbye message and uses:

```
break
```

to stop the `while True` loop.

17. IMPORTANT LIST METHODS AND FUNCTIONS

```
.append() → add an item to the end  
.remove() → remove an item by value  
.pop()    → remove an item by index  
len()     → count items
```

Other list techniques I will encounter later include sorting, reversing, slicing, copying, and more.

18. `.append()` VS `.remove()` VS `.pop()`

```
.append()
```

Adds an item.

```
contacts.append('Chris')
```

```
.remove()
```

Removes a value.

```
contacts.remove('Chris')
```

```
.pop()
```

Removes an item by index.

```
contacts.pop(0)
```

Easy memory trick

```
append → add  
remove → remove by value  
pop    → remove by position/index
```

19. LIST INDEXING

Python lists start counting from index `0`.

Example:

```
contacts = ['Chris', 'Joyce', 'Belle']
```

```
Chris → 0  
Joyce → 1  
Belle → 2
```

This is why a displayed contact number often needs to be converted back into a list index.

20. LISTS CAN BE CHANGED

Lists are **mutable**.

This means their contents can be changed after the list is created.

For example:

```
names = ['Chris', 'Joyce']  
names[1] = 'Belle'
```

Now the list is:

```
['Chris', 'Belle']
```

21. COMMON LIST MISTAKES

Using the wrong index

Remember that the first item is index `0`.

Using `remove()` with an index

This is wrong if you want to remove by position:

```
contacts.remove(2)
```

`remove()` expects a value.

Use:

```
contacts.pop(2)
```

if you want to remove the item at index `2`.

Forgetting that `len()` counts items, not the last index

If a list contains 4 items:

```
len(contacts)
```

returns `4`, but the indexes are:

```
0, 1, 2, 3
```

22. THINGS USED IN MY `lists.py` PROJECT THAT BELONG TO OTHER TOPICS

The project combines Lists with concepts from earlier or future topics.

Functions

```
def clear_screen():
```

Modules

```
import os
```

Loops

```
while True:  
    for number, name in enumerate(...):
```

Decision Making

```
if  
elif  
else
```

Input and conversion

```
input()  
int()
```

String method encountered

```
name.isdigit()
```

This method checks whether a string contains only digits. It has been introduced through the project and can be studied properly later under Strings.

WHAT I CAN DO NOW

After learning Lists and List Operations, I can:

- ✓ Create lists
 - ✓ Create empty lists
 - ✓ Store multiple values in a list
 - ✓ Access list items using indexes
 - ✓ Understand zero-based indexing
 - ✓ Change list items
 - ✓ Add items with `.append()`
 - ✓ Remove items with `.remove()`
 - ✓ Remove items by position with `.pop()`
 - ✓ Count items with `len()`
 - ✓ Loop through lists
 - ✓ Display numbered list items with `enumerate()`
 - ✓ Check whether an item is in a list with `in`
 - ✓ Use lists with user input
 - ✓ Use lists with loops and conditions
 - ✓ Build a contact manager using a list
-

MY PROGRESS

Current Level: Lists and List Operations ✓

Topics Learned

- ✓ Creating lists
- ✓ Accessing list items
- ✓ Changing list items
- ✓ `.append()`
- ✓ `.remove()`
- ✓ `.pop()`
- ✓ `len()`
- ✓ `enumerate()`
- ✓ Looping through lists
- ✓ Checking membership with `in`

Project

- ✓ `lists.py` — Contact Manager
-

LISTS COMPLETE

Lists taught me how to store and manage multiple values in one variable.

I can now create lists, access and change their items, add and remove values, count items, loop through them, and use them to build interactive programs.

My `lists.py` Contact Manager shows how Lists can be combined with loops, conditions, functions, and user input to create a useful program.

 **LISTS** →  **COMPLETE**

Next major topic: FUNCTIONS 